

TCC · 42 PÁGINAS · ORGANIZAÇÃO + BANCO

Controla.AI — Organização do Sistema e Arquitetura do Banco

Documento unificado: pastas, WhatsApp, agente IA, indicadores, deploy Railway/Vercel e o detalhe das **16 tabelas / 18 FK** dos MDs oficiais de modelagem.

Fontes incorporadas

- 1) TCC_DOCUMENTACAO.md — arquitetura, pastas, fluxos, módulos
- 2) CONEXOES_BANCO_DADOS.md — 18 FK, cardinalidade, CASCADE/SET NULL
- 3) ARQUITETURA_BANCO_COMPLETA.md — colunas, ER, dicionário + PNGs



Sumário

Parte A sistema · Parte B banco (MDs)

Fontes: pasta documentacao-tcc/PNGs modelagens banco dados/

Págs.	Bloco	Conteúdo (origem)
1–2	Abertura	Capa e sumário
3–8	Sistema	Visão, camadas, pastas — TCC_DOCUMENTACAO.md
9–16	Módulos	Backend, IA, WhatsApp, frontend, indicadores — TCC_DOCUMENTACAO
17–20	Fluxos/deploy	Zap→gasto, login, Railway, Vercel, env — TCC_DOCUMENTACAO
21–24	Conexões	Como ler FK, 18 ligações, domínios — CONEXOES_BANCO_DADOS.md
25–26	Diagramas PNG	ER visual + detalhes — ARQUITETURA + PNGs
27–28	Modelagem	PK/FK/CASCADE + tabela 18 FK completa
29–36	Dicionário	16 tabelas coluna a coluna — ARQUITETURA_BANCO_COMPLETA.md
37–40	Papel por tabela	Entradas/saídas — CONEXOES
41–42	Fechamento	Mapa arquivos + conclusão

Objetivo

Um único PDF para banca: **como o software se organiza** e **como o PostgreSQL guarda e liga os dados**, sem precisar abrir os três MDs à parte.

A

Visão geral do produto

Fonte: TCC_DOCUMENTACAO.md

O **Controla.AI** é controle financeiro pessoal: o usuário fala no **WhatsApp** (“gastei 45 no almoço”) ou usa o **painel web**; a **OpenAI** interpreta; o **PostgreSQL** grava; o dashboard mostra KPIs e metas.

Camada	Tecnologia	Função
Frontend	React + Vite + Tailwind	Painel, chat IA, metas, admin
Backend	Node + Fastify + TS	API REST, JWT, orquestração
Banco	PostgreSQL Railway + Drizzle	Memória permanente
IA	GPT / Whisper / visão	Parser, áudio, OCR
WhatsApp	Baileys	Canal de entrada

16
tabelas

18
FK

1
hub users

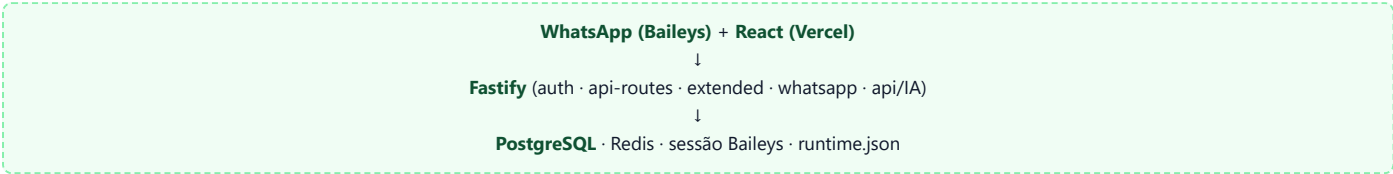
2
clouds

Problema resolvido
Registrar gastos/receitas por texto, áudio, foto ou PDF; categorizar com IA; persistir; exibir indicadores — uma só verdade no banco.

A

Arquitetura em camadas

Canais → Backend → Postgres



Camada	Responsabilidade
Apresentação	SPA — login, dashboard, metas, chat, admin Zap
API REST	Fastify — Zod, JWT, CRUD
Domínio	Parser IA, categorias, metas, orçamento
WhatsApp	QR, sessão, mensagens, keep-alive
OpenAI	GPT, Whisper, visão
Persistência	Drizzle → PostgreSQL

Regra

Tudo que importa financeiramente passa pelo backend e termina no Postgres. Front só exibe; WhatsApp só entrega mensagem.



Estrutura de pastas (raiz)

Princípio: poucas pastas, arquivos comentados

```
controlaai/
├── TCC_DOCUMENTACAO.md      ← doc oficial único
├── documentacao-tcc/        ← PDFs, PNGs ERD, este HTML/PDF
│   └── PNGs modelagens.../  ← 3 MDs + diagramas (fontes deste PDF)
├── frontend/                ← React + funções Vercel
├── backend/
│   ├── src/                 ← Fastify, auth, CRUD, billing
│   ├── api/                 ← agente IA, parser, insights
│   ├── whatsapp/            ← Baileys
│   ├── drizzle/             ← migrations SQL
│   └── .baileys-session/    ← NÃO versionar
├── deploy.ps1 / railway.toml
└── .cursor/rules/          ← atualizar TCC_DOCUMENTACAO.md
```

backend/src

Servidor: index, auth, rotas, schema Drizzle.

backend/api

Cérebro: financial-agent, parser, insights.

backend/whatsapp

Telefone: client, message-handler, bolhas.

frontend

UI + middleware/proxy + auth serverless.

A

Mapa detalhado WhatsApp + API

Arquivos e função (resumo do MD)

Fonte: TCC_DOCUMENTACAO.md — pasta whatsapp/ e api/

Arquivo WhatsApp	Função
client.ts	Socket Baileys, QR, reconexão, anti-replay
message-handler.ts	Pipeline: user → mídia → agente → bolhas
whatsapp-bubbles.ts	Divide em várias mensagens
user-resolver.ts	Telefone BR → users.id
jid-resolver.ts	LID/PN + lid-mapping
inbound-reply-guard / message-dedup	Só responde inbound; sem replay
routes / keep-alive / baileys-log	Admin API, timer 30min, logs

Arquivo API (IA)	Função
financial-agent.ts	Pipeline unificado Zap + web
parser.ts / prompts.ts	FinancialIntent GPT + fallback
insights.ts	"Quanto gastei?", can_spend, KPIs
goal-agent / onboarding / income-*	Metas e renda
media-processor.ts	Whisper + PDF



Fluxo: gasto pelo WhatsApp

Do "oi" ao INSERT

1. Mensagem chega no Baileys (client)
2. Dedup + resolve telefone → users.id
3. Áudio/PDF → media-processor → texto
4. financial-agent → parser (OpenAI/regex)
5. category-resolver + transaction-service INSERT
6. Liga whatsapp_messages.transaction_id (FK 18)
7. Bolhas de confirmação · dashboard lê a mesma linha

Uma verdade

WhatsApp e site usam o **mesmo PostgreSQL**. A FK 18 (transactions → whatsapp_messages) fecha o ciclo mensagem↔gasto — ver CONEXOES.

A

Fluxo: login web e dashboard

Vercel ↔ Railway ↔ Postgres

Login em controlaai-frontend.vercel.app → /api/auth/login (função Vercel ou proxy)
→ bcrypt + JWT 7d → Dashboard → /api/* proxy → Fastify Railway
→ rotas filtram por user_id do token → KPIs (insights / financial-summary)

Na Vercel

SPA + auth/2FA/settings + proxy + relay SMTP.

Na Railway

API 24h + Baileys + Postgres + Redis + Stripe webhook.

BACKEND_URL

no painel Vercel aponta para controlaai-backend-production.up.railway.app. Health: GET /health.

A

Servidor Fastify

index.ts e rotas

Arquivo	Papel
index.ts	Boot: CORS, rotas, health, WhatsApp, admin
auth.ts / mailer.ts	JWT, reset, OTP, 2FA, e-mail SMTP/relay
api-routes.ts	CRUD transações, categorias, budgets
extended-routes.ts	Chat IA, KPIs, metas, imports
governance / billing	LGPD, auditoria, Stripe
db/schema.ts	Espelho TypeScript das 16 tabelas

schema.ts ↔ MD

O que está em **ARQUITETURA_BANCO_COMPLETA.md** é o que o Drizzle declara em backend/src/db/schema.ts e o que o Postgres na Railway armazena.

A

Agente e parser

Mesma lógica no Zap e no chat web

financial-agent orquestra: saudação → onboarding renda → metas → parser gasto/consulta → insights. **parser** devolve JSON FinancialIntent (Zod). Cada chamada OpenAI pode ir para **ai_logs** (tokens, custo).

- Tabelas IA (MD)

ai_conversations · ai_logs · financial_memory · document_imports
- ON DELETE

Maioria CASCADE com users; ai_logs usa SET NULL (histórico pode sobrar).

A

Consultas e KPIs

Números reais do Postgres

queryType	Pergunta	Cálculo
monthly_spending	Quanto gastei?	Soma despesas do mês
can_spend	Posso gastar 500?	Renda – gastos
health_check	Situação?	KPIs agregados
biggest_expense	Maior despesa?	MAX amount
month_comparison	Vs mês passado	Dois períodos

Dashboard: Ganhos / Gastos / Faturamento via fonte única **financial-summary** (lê **transactions** + budgets).

A

Baileys em produção

Por que só na Railway

Baileys precisa de processo **sempre ligado** (não serverless). Sessão em volume (.baileys-session). Estado em **whatsapp_connection** (id=main) — tabela **isolada**, sem FK para users (CONEXOES).

whatsapp_messages Histórico; user_id SET NULL; transaction_id 0:1	whatsapp_sessions JSON por usuário; CASCADE	whatsapp_connection QR/status do número oficial
--	--	--

A

Painel React + edge

pages · lib · api Vercel

Área	Arquivos
Boot	main.tsx · App.tsx · auth · api.ts
Páginas	Dashboard, Goals, AiChat, Settings, admin WhatsApp/LGPD
Edge	middleware.ts · vercel.json rewrites · api/auth/* · proxy

components/ui/* = shaden (terceiros, fora do comentário linha a linha).

A

Produção Railway + Vercel

Variáveis e health

Variável	Onde	Para quê
DATABASE_URL	Railway (+ auth Vercel)	Postgres
JWT_SECRET	Ambos	Assinar token
BACKEND_URL	Vercel	Proxy API
OPENAI_API_KEY	Railway	Parser/Whisper
REDIS_URL	Railway	Cache
ENABLE_WHATSAPP	Railway	Liga Baileys

Validar

/health no Railway · login no front · QR WhatsApp · “gastei 10 teste”.



Como o código liga ao banco

Drizzle · migrations · Railway

schema.ts (TypeScript) → drizzle-kit push / migrations SQL
→ PostgreSQL Railway (banco **railway**)
→ Documentado em ARQUITETURA_BANCO_COMPLETA.md + CONEXOES_BANCO_DADOS.md

As próximas páginas são o conteúdo **detalhado** desses dois MDs (mais a visão do TCC_DOCUMENTACAO sobre o papel do banco), incluindo os **PNGs oficiais** de modelagem.

16

tabelas

18

FK

6

grupos

2

PNGs ER



Como ler conexões e chaves

Fonte: CONEXOES_BANCO_DADOS.md

Símbolos oficiais do MD de conexões

Símbolo	Significado
PK	Chave primária — identifica cada linha
FK	Chave estrangeira — aponta para a PK de outra tabela
1:1	Um em A liga a no máximo um em B
1:N	Um em A liga a vários em B
0:1	Ligação opcional (FK pode ser NULL)
CASCADE	Ao apagar o pai, apaga os filhos
SET NULL	Ao apagar o pai, a FK do filho vira NULL

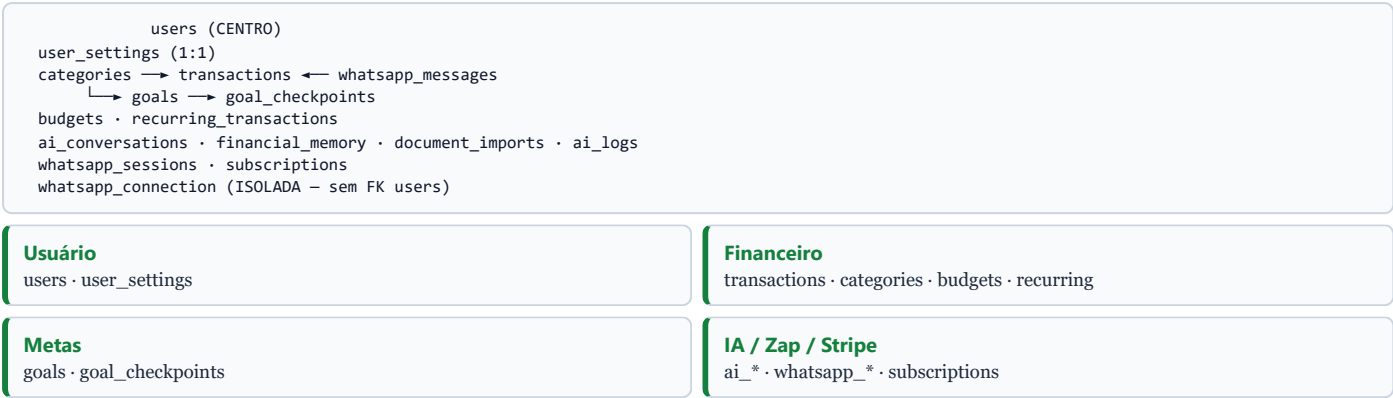
Regra geral

Quase tudo gira em torno de **users**. Apagar um usuário remove a maior parte dos dados dele (CASCADE). Exceções: mensagens/logs podem SET NULL.

B

Mapa rápido — 16 tabelas

users no centro



B

Lista completa das 18 conexões

Parte 1/2 — FKs 1–10

#	De (PK)	Para (FK)	Card.	On delete
1	users.id	user_settings.user_id	1:1	CASCADE
2	users.id	transactions.user_id	1:N	CASCADE
3	users.id	categories.user_id	1:N	CASCADE
4	users.id	goals.user_id	1:N	CASCADE
5	users.id	budgets.user_id	1:N	CASCADE
6	users.id	recurring_transactions.user_id	1:N	CASCADE
7	users.id	ai_conversations.user_id	1:N	CASCADE
8	users.id	financial_memory.user_id	1:N	CASCADE
9	users.id	document_imports.user_id	1:N	CASCADE
10	users.id	whatsapp_messages.user_id	1:N	SET NULL

Das 18, **13 saem de users.id**. Fonte: tabela oficial em CONEXOES_BANCO_DADOS.md.

B

Lista completa das 18 conexões

Parte 2/2 — FKs 11–18

#	De (PK)	Para (FK)	Card.	On delete
11	users.id	whatsapp_sessions.user_id	1:N	CASCADE
12	users.id	subscriptions.user_id	1:N	CASCADE
13	users.id	ai_logs.user_id	1:N	SET NULL
14	categories.id	transactions.category_id	1:N	SET NULL
15	categories.id	goals.category_id	1:N	SET NULL
16	categories.id	recurring_transactions.category_id	1:N	SET NULL
17	goals.id	goal_checkpoints.goal_id	1:N	CASCADE
18	transactions.id	whatsapp_messages.transaction_id	0:1	SET NULL

Exemplo produto

“gastei 45 no almoço” → users (phone) → whatsapp_messages (user_id) → transactions → preenche transaction_id (FK 18) → se houver meta, goal_checkpoints.

B

Conexões por domínio (1/2)

IA · Financeiro · Metas

Domínio	Tabelas	Destaque ON DELETE
IA	ai_conversations, ai_logs, document_imports, financial_memory	ai_logs = SET NULL; demais CASCADE
Financeiro	budgets, categories, recurring, transactions	Apagar categoria = SET NULL no gasto
Metas	goals, goal_checkpoints	Apagar goal = CASCADE nos checkpoints

categories alimenta 3 FKs: transactions, goals e recurring_transactions (SET NULL) — o histórico de valor permanece sem a etiqueta.



Conexões por domínio (2/2)

Usuário · WhatsApp · Assinatura

Domínio	Tabelas	Notas
Usuário	users, user_settings	1:1 settings; users é hub de 13 FKs
WhatsApp	connection, messages, sessions	connection isolada; messages.user_id SET NULL; transaction_id 0:1
Assinatura	subscriptions	CASCADE com users; plan/status Stripe

whatsapp_connection

Única conexão do bot (QR). Não tem user_id — não é “conta de pessoa”, é o aparelho/sessão oficial do sistema.



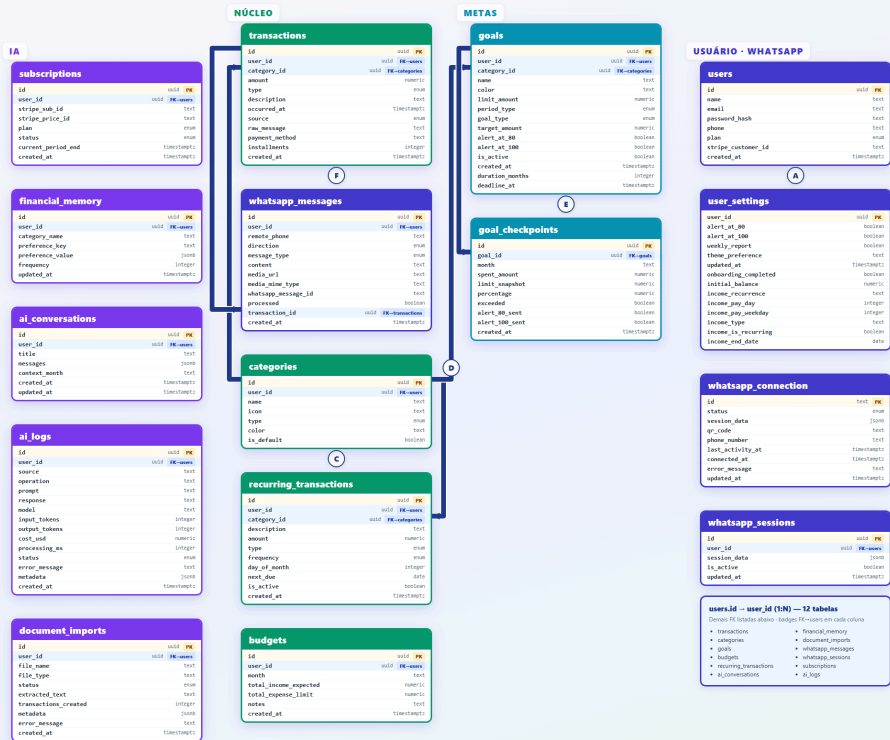
Diagrama ER visual

Fonte: arquitetura-banco-diagrama.png

Arquitetura do banco — tabelas e relacionamentos

16 tabelas · 6 sets A-F (sem cruzamentos) · users id na caixa azul · tabela FK abasado

PK FK A-F - setar - A: user_settings, B: CD category_id, E: goal_id, F: transaction_id



ARQUITETURA_BANCO_COMPLETA.md · pasta PNGs modelagens · PK em destaque

Modelagem + conexões visuais

Fonte: arquitetura-banco-detalhes.png

Arquitectura completa — railway

Controla.AI - Sun Jun 07 2026 20:38:42 GMT-0300 (Horário Padrão de Brasília) - 16 tabelas - todas as colunas - 18 FK

Diagrama — linhas curtas entre tabelas vizinhas

Arquitetura do banco — tabelas e relacionamentos

16 tabelas - 6 setas A-F (sem cruzamento) - users.id na caixa azul + tabela FK abaixo

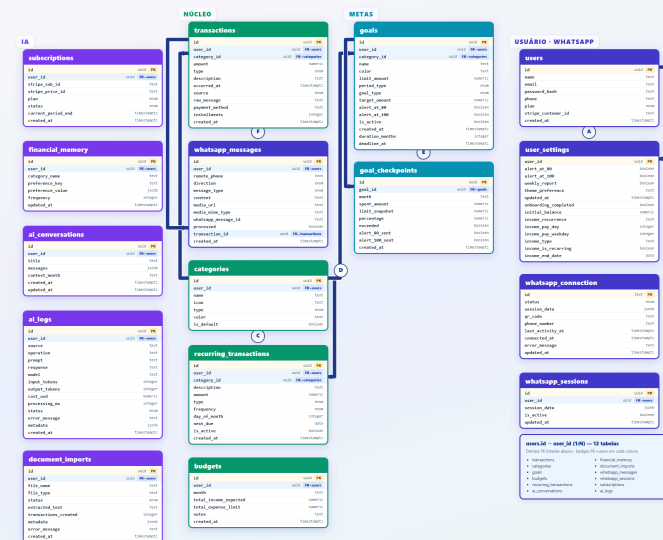


Tabela de relacionamentos (FK)

#	Origins (FK)	Destines (FK)	Card.	On delete
1	users.id	→ user_settings.user_id	1:1	CASCADE
2	users.id	→ transactions.user_id	1:N	CASCADE
3	users.id	→ categories.user_id	1:N	CASCADE
4	users.id	→ goals.user_id	1:N	CASCADE
5	users.id	→ budgets.user_id	1:N	CASCADE
6	users.id	→ recurring_transactions.user_id	1:N	CASCADE
7	users.id	→ ai_conversations.user_id	1:N	CASCADE
8	users.id	→ financial_memory.user_id	1:N	CASCADE
9	users.id	→ document_imports.user_id	1:N	CASCADE
10	users.id	→ whatsapp_messages.user_id	1:N	SET NULL
11	users.id	→ whatsapp_sessions.user_id	1:N	CASCADE
12	users.id	→ subscriptions.user_id	1:N	CASCADE
13	users.id	→ ai_logs.user_id	1:N	SET NULL
14	categories.id	→ transactions.category_id	1:N	SET NULL
15	categories.id	→ goals.category_id	1:N	SET NULL
16	categories.id	→ recurring_transactions.category_id	1:N	SET NULL
17	goals.id	→ goal_checkpoints.goal_id	1:N	CASCADE
18	transactions.id	→ whatsapp_messages.transaction_id	0:1	SET NULL

Mesma modelagem - diagrama + tabela das 18 FK (cardinalidade e ON DELETE)

C

O que a modelagem decide

PK · FK · CASCADE · SET NULL

Chave primária (PK)

“RG” da linha — em geral **id** UUID. Duas linhas nunca compartilham o mesmo id.

CASCADE

Apagar usuário remove gastos, metas e preferências — sem lixo órfão.

Chave estrangeira (FK)

Campo que aponta para o id de outra tabela. Ex.: **transactions.user_id** → **users.id**.

SET NULL

Apagar categoria mantém o gasto; só limpa category_id.

Relacionamentos oficiais (ARQUITETURA §2): 13× users 1:N, 1× user_settings 1:1, 3× categories, 1× goals→checkpoints, 1× transactions→whatsapp_messages 0:1.



Dicionário de dados (1/4)

Colunas condensadas · fonte ARQUITETURA_BANCO_COMPLETA.md

users conta		
Hub central: login, telefone WhatsApp, plano Stripe.		
Coluna	Tipo	Chave
id	uuid	PK
name / email	text	—
password_hash	text	—
phone	text	—
plan	enum	—
stripe_customer_id	text	—
created_at	timestampz	—

transactions gastos/receitas		
Cada lançamento (WhatsApp, web ou PDF).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
category_id	uuid	FK
amount / type	num/enum	—
description	text	—
occurred_at	timestampz	—
source / raw_message	enum/text	—
payment_method / installments	text/int	—

user_settings preferências 1:1		
Alertas, tema, onboarding e perfil de renda mensal.		
Coluna	Tipo	Chave
user_id	uuid	PK/FK
alert_at_80/100	bool	—
weekly_report	bool	—
theme_preference	text	—
onboarding_completed	bool	—
initial_balance	numeric	—
income_*	renda/recorrência	—

categories categorias		
Alimentação, Transporte... padrão ou do usuário.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
name / icon / color	text	—
type	enum	—
is_default	bool	—

C

Dicionário de dados (2/4)

Colunas condensadas · fonte ARQUITETURA_BANCO_COMPLETA.md

budgets orçamento mês		
Renda esperada e limite de gastos do mês.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
month	text	—
total_income_expected	numeric	—
total_expense_limit	numeric	—
notes	text	—

goals metas		
Limite ou alvo com prazo (duration_months / deadline_at).		
Coluna	Tipo	Chave
id	uuid	PK
user_id / category_id	uuid FK	—
name / limit_amount / target_amount	text/num	—
period_type / goal_type	enum	—
alert_at_80/100 / is_active	bool	—
duration_months / deadline_at	int/ts	—

recurring_transactions recorrentes		
Aluguel, salário fixo — próxima data e frequência.		
Coluna	Tipo	Chave
id	uuid	PK
user_id / category_id	uuid FK	—
description / amount / type	text/num/enum	—
frequency / day_of_month / next_due	enum/int/date	—
is_active	bool	—

goal_checkpoints progresso meta		
Snapshot mensal, % e flags de alerta 80%/100%.		
Coluna	Tipo	Chave
id	uuid	PK
goal_id	uuid	FK
month / spent_amount / limit_snapshot	text/num	—
percentage / exceeded	num/bool	—
alert_80_sent / alert_100_sent	bool	—

 Dicionário de dados (3/4)
Colunas condensadas · fonte ARQUITETURA_BANCO_COMPLETA.md

whatsapp_messages msgs Zap		
Inbound/outbound; pode ligar a um gasto (transaction_id).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
remote_phone / direction / message_type	text/enum	—
content / media_*	text	—
whatsapp_message_id	text	—
processed	bool	—
transaction_id	uuid	FK

whatsapp_connection número oficial		
QR/status do bot — SEM FK para users.		
Coluna	Tipo	Chave
id	text (=main)	PK
status	enum	—
session_data / qr_code	jsonb/text	—
phone_number	text	—
last_activity_at / connected_at	timestampz	—
error_message	text	—

whatsapp_sessions sessão user Zap		
Estado conversacional por usuário (JSON).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
session_data	jsonb	—
is_active	bool	—

ai_conversations chat web		
Histórico de mensagens do painel (JSON).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
title / messages / context_month	text/jsonb	—

C

Dicionário de dados (4/4)

Colunas condensadas · fonte ARQUITETURA_BANCO_COMPLETA.md

ai_logs auditoria IA		
Tokens, custo USD, modelo, status de cada chamada OpenAI.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
source / operation / model	text	—
input/output_tokens / cost_usd	int/num	—
processing_ms / status / error_message	int/enum/text	—
metadata	jsonb	—

document_imports PDF/extrato		
Importação de documentos pelo painel + texto extraído.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
file_name / file_type / status	text/enum	—
extracted_text / transactions_created	text/int	—
metadata / error_message	jsonb/text	—

financial_memory memória IA		
Preferências aprendidas (categoria favorita, etc.).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
category_name / preference_key	text	—
preference_value / frequency	jsonb/int	—

subscriptions Stripe		
Assinatura/plano pago espelhado do Stripe.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK
stripe_sub_id / stripe_price_id	text	—
plan / status	enum	—
current_period_end	timestampz	—

B

Papel de cada tabela (1/2)

Fonte: CONEXOES_BANCO_DADOS.md — “Cada tabela”

Tabela	Domínio	Papel	FK principal
ai_conversations	IA	Histórico do chat web com a IA (JSON messages).	user_id → users (CASCADE)
ai_logs	IA	Log de chamadas OpenAI (tokens, custo, status).	user_id → users (SET NULL)
budgets	Financeiro	Orçamento mensal: renda esperada e limite.	user_id → users (CASCADE)
categories	Financeiro	Categorias receita/despesa (padrão ou user).	user_id; sai para tx/goals/recurring
document_imports	IA	Importação de PDFs/extratos pelo painel.	user_id → users (CASCADE)
financial_memory	IA	Preferências aprendidas (JSON por chave).	user_id → users (CASCADE)
goal_checkpoints	Metas	Progresso mensal e alertas 80%/100%.	goal_id → goals (CASCADE)
goals	Metas	Meta de limite/alvo com prazo.	user_id + category_id

B

Papel de cada tabela (2/2)

Fonte: CONEXOES_BANCO_DADOS.md

Tabela	Domínio	Papel	FK principal
recurring_transactions	Financeiro	Lançamentos recorrentes (salário, aluguel).	user_id + category_id
subscriptions	Stripe	Assinatura/plano pago.	user_id → users (CASCADE)
transactions	Financeiro	Cada gasto/receita do usuário.	user_id + category_id; alvo da FK 18
user_settings	Usuário	Preferências 1:1 + perfil de renda.	user_id PK/FK (CASCADE)
users	Usuário	Conta: e-mail, hash, phone, plan.	Hub — 13 saídas
whatsapp_connection	Zap	Número oficial do bot (QR/status).	Nenhuma (isolada)
whatsapp_messages	Zap	Mensagens; opcionalmente ligadas a gasto.	user_id SET NULL; transaction_id 0:1
whatsapp_sessions	Zap	Estado conversacional por usuário.	user_id → users (CASCADE)

C

Detalhe de colunas — conta

Transcrição do MD ARQUITETURA §3

users		
Coluna	Tipo	Chave
id	uuid	PK
name / email	text	email UNIQUE
password_hash	text	bcrypt
phone	text	lookup WhatsApp
plan / stripe_customer_id	enum / text	—
created_at	timestampz	—

user_settings		
Coluna	Tipo	Nota
user_id	uuid	PK/FK 1:1
alert_at_80 / alert_at_100 / weekly_report	bool	notificações
theme_preference / onboarding_completed	text/bool	UI + fluxo inicial
initial_balance / income_*	num/text/int	perfil de renda do agente

 Detalhe de colunas — financeiro

ARQUITETURA_BANCO_COMPLETA.md §3

transactions		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK CASCADE
category_id	uuid	FK SET NULL
amount / type	numeric / enum	expense income ...
description / occurred_at	text / ts	—
source / raw_message	enum / text	whatsapp web pdf
payment_method / installments	text / int	opcional
categories		
id / user_id / name / icon / type / color / is_default		uuid, FK opcional, textos, enum, bool

C

Metas e WhatsApp — colunas-chave

ARQUITETURA §3

Tabela	Colunas essenciais
goals	limit_amount, target_amount, period_type, goal_type, alert_at_80/100, duration_months, deadline_at, category_id
goal_checkpoints	month, spent_amount, limit_snapshot, percentage, exceeded, alert_80_sent, alert_100_sent
whatsapp_messages	remote_phone, direction, message_type, content, media_*, whatsapp_message_id, processed, transaction_id
whatsapp_connection	id=main, status, session_data, qr_code, phone_number, last_activity_at, error_message
ai_logs	source, operation, model, tokens, cost_usd, processing_ms, status, metadata
subscriptions	stripe_sub_id, stripe_price_id, plan, status, current_period_end

Completo no MD

Todas as linhas Null/Tipo/Chave estão em **ARQUITETURA_BANCO_COMPLETA.md** §3; este PDF resume o essencial para a banca.



Autenticação e o Postgres

users · JWT · LGPD

Cadastro grava **users** + **user_settings** + consentimentos LGPD. Senha só como **password_hash** (bcrypt). Login emite JWT com **user_id**; toda rota financeira filtra por esse id — uma conta nunca vê a outra.

UNIQUE email

Impede dois cadastros com o mesmo e-mail (regra no Postgres).

phone

Chave prática do WhatsApp (variantes com/sem 9º dígito no resolver).

De onde vêm os números da tela
Dashboard e insights

Indicador na UI	Origem no banco
Gastos / Ganhos	transactions filtrado por type + período
Saldo / orçamento	budgets + soma de transactions
Progresso de meta	goals + goal_checkpoints
Gráfico por categoria	JOIN transactions ↔ categories
Custo IA (admin)	ai_logs.cost_usd



Ciclo WhatsApp completo no banco

CONEXOES + TCC_DOCUMENTACAO

```
whatsapp_connection (bot online)
↓ mensagem
whatsapp_messages (user_id, content, processed=false)
↓ agente
transactions (amount, category_id, source=whatsapp, raw_message)
↓
whatsapp_messages.transaction_id = transactions.id (FK 18, SET NULL)
↓ opcional
goal_checkpoints atualizado se houver meta da categoria
```

A

Mapa de arquivos de aplicação
MAPA-SISTEMA backend/frontend

Backend src/* · api/* (~27) · whatsapp/* — cabeçalho Doc TCC + comentários PT. Exceto seeds.	Frontend pages, lib, components app, hooks, api/*, middleware. Exceto components/ui/*.
---	---

package.json documentado em _tcc. **package-lock.json** não se edita (gerado pelo npm). **vercel.json** documentado no middleware (JSON sem comentário).



Fontes deste PDF

Caminhos no repositório

Arquivo	O que alimentou neste PDF
.../TCC_DOCUMENTACAO.md	Visão, pastas, módulos Zap/IA, fluxos, deploy, mapa
.../CONEXOES_BANCO_DADOS.md	Símbolos PK/FK, 18 ligações, domínios, papel por tabela
.../ARQUITETURA_BANCO_COMPLETA.md	ER, relacionamentos, colunas, índice PNG
arquitetura-banco-*.png	Páginas 22–23 (imagens embutidas)

Manutenção

Alterou schema → atualize os 2 MDs de banco + **TCC_DOCUMENTACAO.md** (raiz) + regenere este HTML/PDF.



O que saber na apresentação

Sistema + banco

#	Pergunta típica	Resposta curta
1	Onde roda?	Front Vercel · API/Zap/Postgres Railway
2	Centro do banco?	users — 13 FKs saem dele
3	Como o Zap vira gasto?	mensagem → agente → transactions → FK 18
4	CASCADE vs SET NULL?	Apaga filhos × só limpa vínculo
5	Por que Zap na Railway?	Baileys precisa de processo contínuo
6	Onde está o schema?	schema.ts = ARQUITETURA MD = Postgres

✓

Números do Controla.AI
Espelho dos MDs

16

tabelas Postgres

18

chaves FK

6

domínios

1

hub users

Software
React · Fastify · Baileys · OpenAI · Drizzle

Cloud
Vercel (UI) · Railway (API+DB+Redis+Zap)

Documento vivo
Este PDF foi gerado por generate-ORGANIZACAO-SISTEMA-html.ts + ...-pdf.ts incorporando os três MDs da pasta de modelagens.



Organização do software

TCC_DOCUMENTACAO.md

O Controla.AI separa claramente: **frontend** (experiência), **src** (API e segurança), **api** (inteligência financeira), **whatsapp** (canal Baileys). Em produção, a Vercel recebe o usuário; a Railway executa o trabalho pesado e fala com o Postgres.

Lembrete

Código de aplicação comentado em português; MAPA-SISTEMA lista os arquivos; doc oficial na raiz.